

# Electronica Store

Especificação do projeto · gerado em 23/08/2026

52

TAREFAS CONCLUÍDAS

0

EM ABERTO

7

FASES

Marketplace de eletrônicos em React, no espírito de Shopee / Mercado Livre. Catálogo grande, vendedores, avaliações, cupons, checkout completo e área do usuário. **Tudo com dados mocados** — não existe backend.

A identidade visual segue a **Shopee**: fundo claro, laranja como cor de marca, cards brancos densos e tipografia enxuta. Isso **substitui** o tema escuro atual (night/volt/fuchsia, superfícies em vidro) — não convive com ele.

## Identidade visual (Shopee)

Tokens em `src/index.css`, como variáveis CSS e utilitários Tailwind.

| Token                                                 | Valor                                       | Uso                                               |
|-------------------------------------------------------|---------------------------------------------|---------------------------------------------------|
| <code>--brand</code>                                  | <code>#EE4D2D</code>                        | laranja da marca: preços, CTAs, ativos            |
| <code>--brand-dark</code>                             | <code>#D73211</code>                        | hover de botões primários                         |
| <code>--brand-soft</code>                             | <code>#FFF0ED</code>                        | fundos de destaque suave                          |
| <code>--header-from</code> / <code>--header-to</code> | <code>#F53D2D</code> → <code>#FF6633</code> | gradiente do topo                                 |
| <code>--page</code>                                   | <code>#F5F5F5</code>                        | fundo da página                                   |
| <code>--surface</code>                                | <code>#FFFFFF</code>                        | cards e painéis                                   |
| <code>--line</code>                                   | <code>#E5E5E5</code>                        | bordas de 1px                                     |
| <code>--ink</code>                                    | <code>#222222</code>                        | texto principal                                   |
| <code>--ink-soft</code>                               | <code>#757575</code>                        | texto secundário                                  |
| <code>--star</code>                                   | <code>#FFCE3D</code>                        | estrelas de avaliação                             |
| <code>--tag</code>                                    | <code>#FFD839</code>                        | fita de desconto (texto em <code>--brand</code> ) |
| <code>--ship</code>                                   | <code>#00BFA5</code>                        | selo de frete grátis                              |

Regras de composição:

- Fundo da página cinza, conteúdo em cards brancos com borda de 1px. Sombra quase nula em repouso; ao passar o mouse, o card sobe levemente e ganha sombra.
- Cabeçalho fixo com gradiente laranja: linha superior de links pequenos, e abaixo logo + busca (input branco arredondado com botão laranja) + carrinho.

- Cartão de produto: imagem quadrada, nome em 2 linhas com reticências, preço em laranja com o "R\$" menor, estrelas e quantidade vendida em texto miúdo.
- Fita de desconto no canto superior direito da imagem: fundo `--tag`, texto `--brand`, formato "-25%".
- Selos pequenos e retangulares (não pílulas): "Frete Grátis", "Mais vendido".
- Grade densa: 6 colunas em telas grandes, 2 no celular.
- Cantos suaves (2 a 4px), nunca totalmente arredondados, exceto a busca.
- Sem glassmorphism, sem gradientes coloridos fora do cabeçalho e dos botões.

## Stack

| Camada   | Escolha                                              |
|----------|------------------------------------------------------|
| UI       | React 19 + TypeScript                                |
| Build    | Vite 8                                               |
| Estilo   | Tailwind CSS 4 (via <code>@tailwindcss/vite</code> ) |
| Rotas    | React Router 7                                       |
| Animação | Framer Motion 13                                     |
| Lint     | Oxlint                                               |

Persistência é `localStorage`. Latência de rede é simulada quando fizer sentido.

## Comandos

```
npm run dev      # servidor de desenvolvimento
npm run build    # tsc -b && vite build
npm run lint     # oxlint
```

## Estrutura

```
src/
  main.tsx          ponto de entrada
  App.tsx           rotas + providers (Auth > Favorites > Cart > Compare)
  index.css         tema, tokens, .card/.btn-brand, foco visível
  data/            dados mocados e tipos (exportados de onde nascem)
    products.ts     catálogo (60 itens), categorias, getProduct
    sellers.ts      vendedores, getSeller
    reviews.ts     avaliações, reviewsFor / ratingSummary
    coupons.ts      cupons, getCoupon
  lib/             regras de domínio e persistência (localStorage)
    format.ts       formatBRL / formatDate / formatCompact / formatInstallments
    catalog.ts      estado do catálogo, filtros e ordenação
    shipping.ts     quoteShipping e frete grátis (acima de R$ 999)
    totals.ts       computeCoupon e desconto Pix
    orders.ts       pedidos (createOrder, getOrder/getOrders)
    addresses.ts    endereços salvos
    recent.ts       produtos vistos recentemente
    useLoading.ts   useSimulatedLoading (latência simulada)
  context/         estado global por domínio (XxxCore = dados+hook, XxxContext = provider)
    cartCore.ts / CartContext.tsx      carrinho, seleção, cupom
    favoritesCore.ts / FavoritesContext.tsx favoritos
    authCore.ts / AuthContext.tsx      login mock (usuário)
    compareCore.ts / CompareContext.tsx comparador (até 3)
  components/      componentes reutilizáveis
    Navbar, Footer, Reveal, SmartImage
    ProductCard, ProductReviews, SellerBlock, Questions
    CatalogView, FilterPanel, Pagination
    RecentlyViewed, Skeleton, FlashSale, CompareBar
  pages/           uma página por rota
```

## Rotas

| Rota             | Página                   |
|------------------|--------------------------|
| /                | Home                     |
| /produtos        | catálogo completo        |
| /busca           | busca por termo          |
| /categoria/:slug | catálogo por categoria   |
| /produto/:id     | detalhe do produto       |
| /loja/:id        | loja do vendedor         |
| /carrinho        | carrinho                 |
| /checkout        | checkout                 |
| /pedido/:id      | confirmação do pedido    |
| /favoritos       | favoritos                |
| /entrar          | login (mock)             |
| /pedidos         | meus pedidos             |
| /pedidos/:id     | acompanhamento do pedido |
| /enderecos       | endereços salvos         |
| /comparar        | comparador de produtos   |
| /sobre           | sobre a loja             |
| *                | 404 (não encontrado)     |

## Convenções

- Componentes em PascalCase, um por arquivo, `export default`.
- Texto da interface em **português**.
- Preços em BRL via `Intl.NumberFormat("pt-BR", { style: "currency", currency: "BRL" })`.
- Datas via `Intl.DateTimeFormat("pt-BR")`.
- Só classes Tailwind. Utilitários compartilhados ficam em `index.css`.
- Cada contexto novo entra em `src/context/` e é montado em `App.tsx`.
- Persistência sempre dentro de `try/catch`, com chave prefixada `electronica:`.
- **Nenhuma dependência nova**. O que precisar, implemente à mão.
- Tipos exportados do módulo onde o dado nasce.

## Backlog

---

Uma tarefa por iteração, de cima para baixo. Ao concluir, troque [ ] por [x].

### Fase 0 — Identidade visual Shopee

Vem primeiro: todo componente criado depois já nasce no visual certo.

- ✓ **Reescrever os tokens em `src/index.css`**. Trocar o tema escuro pelos tokens da tabela "Identidade visual" acima, como variáveis CSS em `:root`. Definir `body` com fundo `--page` e texto `--ink`. Remover `.glass` e `.text-gradient`. Criar `.card` (fundo branco, borda 1px `--line`, raio 2px, transição de sombra) e `.btn-brand` (fundo `--brand`, texto branco, hover `--brand-dark`).
- ✓ **Refazer o `Navbar.tsx` no padrão Shopee**: cabeçalho fixo com gradiente `--header-from` → `--header-to`, linha superior com links pequenos (Vender, Central de Ajuda, Notificações), e linha principal com logo à esquerda, busca centralizada (input branco arredondado + botão laranja com ícone de lupa) e ícone de carrinho com contador à direita.
- ✓ **Refazer o `ProductCard.tsx` no padrão Shopee**: card branco, imagem quadrada no topo, fita de desconto no canto superior direito quando houver `oldPrice`, nome em 2 linhas com reticências, preço em laranja com "R\$" menor, linha inferior com estrelas e "N vendidos". Elevação sutil no hover.
- ✓ **Converter a `Home.tsx` para o novo tema**: faixa de categorias em grade com ícones, banner promocional laranja, e seções de produtos em grade densa. Remover animações de vidro e gradientes coloridos que não sejam da marca.
- ✓ **Converter `Products.tsx`, `ProductDetail.tsx`, `Cart.tsx` e `About.tsx` para o novo tema**: fundo cinza, blocos em cards brancos, botões `--brand`, filtros e selects claros com borda de 1px.
- ✓ **Converter o `Footer.tsx`**: fundo branco com borda superior, colunas de links em texto miúdo cinza, e faixa inferior com direitos reservados.
- ✓ **Varredura final do tema escuro**: procurar por classes remanescentes (`night-`, `volt-`, `fuchsia-`, `glass`, `text-gradient`, `bg-white/5`, `border-white/10`, `text-slate-`) em todo o `src/` e substituir pelos tokens novos. O build precisa passar e nenhuma tela pode ficar com fundo escuro.

### Fase 1 — Fundação de dados

- ✓ **Expandir o catálogo para 60 produtos** em `src/data/products.ts`, distribuídos entre as categorias existentes. Cada um com nome realista, preço, `oldPrice` quando houver desconto, rating entre 3.5 e 5, número de reviews, galeria com 3 imagens e 3 a 5 destaques.
- ✓ **Adicionar campos de marketplace ao `Product`**: `sellerId`, `stock`, `sold` (unidades vendidas), `freeShipping: boolean`, `condition: "novo" | "usado"`, `installments: { count: number; value: number }`. Preencher em todos os produtos.
- ✓ **Criar `src/data/sellers.ts` com 8 vendedores** mocados: `id`, `name`, `logo`, `rating`, `sales`, `since` (ano), `location`, `isOfficial: boolean`. Ligar aos produtos pelo `sellerId`.

- ✓ **Criar** `src/data/reviews.ts` com ~120 avaliações mocadas espalhadas entre os produtos: `id`, `productId`, `author`, `rating`, `date`, `comment`, `helpful` (contagem), `photos?: string[]`.
- ✓ **Criar** `src/data/coupons.ts` com 6 cupons: `code`, `type` (`percent` | `fixed` | `freeship`), `value`, `minValue`, `expiresAt`.
- ✓ **Criar** `src/lib/format.ts` com `formatBRL`, `formatDate`, `formatCompact` (1,2 mil) e `formatInstallments`. Trocar todas as formatações espalhadas pelo projeto por essas funções.

## Fase 2 — Carrinho e compra

- ✓ **Persistir o carrinho no localStorage** (`electronica:cart`), lendo no estado inicial e gravando a cada mudança, dentro de `try/catch`.
- ✓ **Corrigir o aviso do lint em** `CartContext.tsx` — remover o reexport de `FALLBACK_IMAGE` e importar direto de `data/products` onde for usado. `npm run lint` deve ficar sem avisos.
- ✓ **Agrupar o carrinho por vendedor**, como na Shopee: cada bloco com nome do vendedor, seus itens e subtotal próprio.
- ✓ **Seleção de itens no carrinho**: checkbox por item e por vendedor, "selecionar todos", e o total considerando apenas os selecionados.
- ✓ **Campo de cupom no carrinho**: valida contra `data/coupons.ts`, aplica o desconto, mostra erro para código inválido ou valor mínimo não atingido.
- ✓ **Frete por CEP simulado**: campo de CEP no carrinho, cálculo determinístico a partir dos dígitos, exibindo prazo e valor. Frete grátis acima de R\$ 999 ou quando todos os itens forem `freeShipping`.
- ✓ **Barra de progresso de frete grátis**: mostra quanto falta para atingir R\$ 999, com barra animada.
- ✓ **Página de checkout** em `/checkout` (`src/pages/Checkout.tsx`), em etapas: endereço, entrega, pagamento e revisão. Navegação entre etapas com validação.
- ✓ **Formas de pagamento no checkout**: Pix (com desconto de 5% e QR fake), boleto, e cartão com seleção de parcelas. Refletir no total.
- ✓ **Confirmação de pedido** em `/pedido/:id`: número gerado, resumo, prazo estimado e status inicial. Limpar o carrinho ao concluir.

## Fase 3 — Descoberta

- ✓ **Busca com sugestões**: no `Navbar`, dropdown que sugere produtos e categorias enquanto digita, com navegação por teclado (setas e Enter).
- ✓ **Página de resultados de busca** em `/busca?q=`, com contagem de resultados e mensagem de vazio.

- ✓ **Filtros avançados no catálogo:** faixa de preço, avaliação mínima, condição, frete grátis e vendedor oficial. Refletidos na URL como query params.
- ✓ **Paginação do catálogo:** 12 por página, controles de navegação e preservação dos filtros na URL.
- ✓ **Ordenação por mais vendidos** somando à ordenação existente, usando o campo `sold`.
- ✓ **Página de categoria** em `/categoria/:slug`, com banner próprio e os filtros já aplicados àquela categoria.
- ✓ **Vistos recentemente:** registra os últimos 8 produtos abertos em `localStorage` e exibe uma faixa na Home e no `ProductDetail`.

## Fase 4 — Produto

- ✓ **Galeria com miniaturas** no `ProductDetail`: coluna de thumbs, troca da imagem principal e zoom ao passar o mouse.
- ✓ **Bloco do vendedor** no `ProductDetail`: logo, nome, reputação, vendas, selo de oficial e link para a loja.
- ✓ **Avaliações** no `ProductDetail`: nota média, histograma de 1 a 5 estrelas, lista paginada com filtro por nota e botão "útil".
- ✓ **Perguntas e respostas:** bloco com perguntas mocadas, respostas do vendedor e campo para enviar (adiciona à lista em memória).
- ✓ **Produtos relacionados:** até 8 da mesma categoria, em carrossel horizontal.
- ✓ **Indicadores de estoque e urgência:** "últimas N unidades" quando o estoque for baixo, e contagem de vendidos.
- ✓ **Página do vendedor** em `/loja/:id`: cabeçalho com reputação e grade dos produtos daquele vendedor.

## Fase 5 — Área do usuário

- ✓ **Contexto de favoritos** persistido, com botão de coração no `ProductCard` e no `ProductDetail`, e contador no `Navbar`.
- ✓ **Página de favoritos** em `/favoritos`, com opção de mover para o carrinho.
- ✓ **Login mocado:** contexto de usuário, página `/entrar` que aceita qualquer e-mail, persiste a sessão e troca o `Navbar` para o nome do usuário.
- ✓ **Meus pedidos** em `/pedidos`: lista os pedidos gravados no `localStorage` com status, data e valor, e link para o detalhe.

- ✓ **Rastreamento do pedido:** linha do tempo com etapas (confirmado, preparando, enviado, em trânsito, entregue), avançando conforme a data.
- ✓ **Endereços salvos** em `/enderecos` : adicionar, editar, remover e definir o principal, tudo persistido.
- ✓ **Notificações:** sino no `Navbar` com dropdown de avisos mocados e contador de não lidas.

## Fase 6 — Acabamento

- ✓ **Página 404** ( `src/pages/NotFound.tsx` ) no visual do site, e rota `*` apontando para ela.
- ✓ **Skeletons de carregamento** para as grades de produtos e para o detalhe, exibidos durante uma latência simulada curta.
- ✓ **Ofertas relâmpago na Home:** faixa com contador regressivo real e produtos com desconto.
- ✓ **Comparador de produtos:** selecionar até 3 do catálogo e comparar specs lado a lado em `/comparar` .
- ✓ **Responsividade móvel:** revisar `Navbar` (menu hambúrguer), filtros do catálogo (drawer) e carrinho em telas pequenas.
- ✓ **Acessibilidade:** `aria-label` em todo botão só de ícone, `alt` descritivo, foco visível, navegação por teclado nos dropdowns e modais.
- ✓ **Metadados:** `<title>` , `description` , Open Graph, `lang="pt-BR"` e `theme-color` no `index.html` .
- ✓ **Atualizar este arquivo:** revisar Estrutura e Rotas para refletir tudo que passou a existir.

## Regras do loop

---

1. Execute **uma** tarefa por iteração — a primeira não marcada. 2. Leia os arquivos que vai alterar antes de editar. 3. Depois de editar, rode `npm run build` . Se falhar, conserte antes de encerrar. 4. Só marque `[x]` quando a tarefa estiver concluída e o build passar. 5. Não instale dependências novas. 6. Não reescreva o que já funciona; faça a menor mudança que resolve. 7. Interface em português.